AI基础

01-数学基础

01-1 Linear Algebra

01-2 Regression

> **Question**
>
> 在一组数据点中找到一个多项式函数，拟合这组数据 (Linear/Polynomial Regression)
> 数据为 $\mathbf{x} = (x_1, \ldots, x_N)^T, \mathbf{t} = (t_1, \ldots, t_N)^T$
> 求"最佳"参数$\mathbf{w} = (w_0, \ldots, w_M)^T$

令M次多项式函数$y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j x^j$，误差函数$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(y(x_n, \mathbf{w}) - t_n)^2$，"最佳拟合"即为使得$E(\mathbb{w})$最小的$\mathbb{w}$

> **是不是$\mathbb{w}$越大越好呢?**
>
> 多项式次数过高会导致**过拟合**
> 训练数据过少，参数过多可能会**过拟合**

- 过拟合解决方案:
  - 增加数据量
  - 权重正则化

01-3 Probability Theory

CS229-Mathmatical Fundamentals

[CS229-Probability Review]

概率的定义

- 概率三要素
  - 样本空间Ω
  - 事件空间$\mathcal{F} \subset 2^{\Omega}$
  - 概率测度$P : \mathcal{F} \to [0, 1]$
- 联合概率:
  - $X = x_i$ 和 $Y = y_j$同时发生的概率

    - Sum Rule
        - $P(X=x_i)=\sum_{j=1}^{L}P(X=x_i,Y=y_j)$
- 条件概率
    - 在事件$X=x_i$发生的情况下，事件$Y=y_j$发生的概率
    - Product Rule
        - $P(X=x_i,Y=y_j)=P(Y=y_j|X=x_i)P(X=x_i)$

Beyes' theorem

$$P(Y|X)=\frac{P(X|Y)P(Y)}{\sum_Y P(X|Y)P(Y)}$$

- PDF与CDF
  - $f_X(x)=\frac{\mathrm{d}F_X(x)}{\mathrm{d}x}$
  - 所以$F_X(k)=\int_{-\infty}^{k}F_X(x)\mathrm{d}x$
  - 且
  $P(x\le X\le x+\Delta x)=F_X(x+\Delta x)-F_X(x)=\int_x^{x+\Delta x}f_X(x)\mathrm{d}x=f_X(\xi)\Delta x\approx f_X(x)\Delta x$
  感觉不如看CS229-Probability

02-机器学习-线性模型Part01

任务

- 有监督学习
    - 分类问题、回归问题…
- 无监督学习
    - 聚类、密度估计、降维…
- 半监督学习
- 弱监督学习
- 强化学习

经验

- 训练数据
- 测试数据

表现

- 表现该如何衡量?
  - 损失函数$loss=\mathcal{L}(f,\mathbb{D})$
  目标：构建预测器$f:\mathbb{X}\to\mathbb{Y}$，能够做出合理预测
  途径：最小化损失函数的期望$\mathbb{E}[\mathcal{L}(f,\mathbb{D})]$

线性回归

> **Note**
>
> 连续数值预测：回归问题
> 离散数值预测：分类问题

目标：构建一个预测器$f:\mathbb{X}\to\mathbb{Y}$来最小化损失函数
线性模型：$f(\mathbf{x})=\sum_{i=1}^{d}w_ix_i+\mathbf{b}=\mathbf{w}^\top\mathbf{x}+\mathbf{b}$
给定数据集：$\mathbb{D}=\{(\mathbf{x_1},y_1),(\mathbf{x_2},y_2),\cdots,(\mathbf{x_n},y_n)\}$，求$\mathbf{w}$使得

$$\mathcal{L}(\mathbf{w},\mathbb{D})=E[(f(\mathbf{x})-y)^2]$$

最小

由大数定理，我们使用 经验均值/Empirical Mean 估计期望：

$$\begin{aligned}l(\mathbf{w},\mathbb{D})&=\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}(f(\mathbf{x})-y)^2\\&=\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}\left(y-\mathbf{w}^\top\mathbf{x}-\mathbf{b}\right)^2\end{aligned}$$

从而

$$\begin{aligned}(\mathbf{w}^*,b^*)&=\arg\min_{(\mathbf{w},b)}\frac{1}{|\mathbb{D}|}\sum_{(\mathbf{x},y)\in\mathbb{D}}\left(y-\mathbf{w}^\top\mathbf{x}-\mathbf{b}\right)^2\\&=\frac{1}{|\mathbb{D}|}\arg\min_{\beta}\sum_{i=1}^{n}(\mathbf{X}_i\beta-y_i)^2\\&=\frac{1}{|\mathbb{D}|}\arg\min_{\beta}\|\mathbf{A}\beta-\mathbf{y}\|^2\end{aligned}$$

其中

$$\mathbf{X}_i^\top=\begin{pmatrix}x_i^{(1)}\\\vdots\\x_i^{(d)}\\1\end{pmatrix}\qquad\mathbf{A}=\begin{pmatrix}\mathbf{X}_1\\\vdots\\\mathbf{X}_n\end{pmatrix}\qquad\beta=\begin{pmatrix}w_1\\\vdots\\w_n\\b\end{pmatrix}\qquad\mathbf{y}=\begin{pmatrix}\mathbf{y}_1\\\vdots\\\mathbf{y}_n\end{pmatrix}$$

令函数$\mathcal{J}(\beta)=\|\mathbf{A}\beta-\mathbf{Y}\|^2=(\mathbf{A}\beta-\mathbf{Y})^\top(\mathbf{A}\beta-\mathbf{Y})=\beta^\top\mathbf{A}^\top\mathbf{A}\beta-\mathbf{Y}^\top\mathbf{A}\beta-\beta^\top\mathbf{A}^\top\mathbf{Y}+\mathbf{Y}^\top\mathbf{Y}$
求导：

$$\frac{\partial\mathcal{J}}{\partial\beta}|_{\hat\beta}=2\mathbf{A}^\top\mathbf{A}\hat\beta-2\mathbf{A}^\top\mathbf{y}=0$$

即

$$\mathbf{A}^\top\mathbf{A}\hat\beta=\mathbf{A}^\top\mathbf{y}$$

当$\mathbf{A}^\top\mathbf{A}$可逆时，可以计算

$$\widehat{\beta} = \left(\mathbf{A}^\top\mathbf{A}\right)^{-1}\mathbf{A}^\top\mathbf{y}$$

什么时候可逆？什么时候不可逆？

- 样本数量 < 特征维度时不可逆（方程组欠定）
  - 不可逆时使用正则化/Regularization

但是，当矩阵大小非常大时候，即使$\mathbf{A}^\top\mathbf{A}$可逆，求解矩阵的逆开销也是非常大的
有没有什么更好的方法？

> 考虑优化问题！

梯度下降法

求$J(\beta)$的最小值，由于它是convex function，可以通过计算梯度的方法迭代
Step1. Initialize: β_0
Step2. Update:

$$\beta_{t+1} = \beta_t - \frac{\alpha}{2}\frac{\partial \mathcal{J}}{\partial \beta}|_{\beta_t}$$

Note

Step-size α的作用：

- 这个数值的大小代表学习率/learning rate
- small α：收敛速度慢，但不容易在最值附近摆动
- large α：收敛速度快，但有可能摆动

Stop. Convergence Condition:

$$\frac{\partial \mathcal{J}}{\partial \beta}|_{\beta_t} < \epsilon$$

按照该算法计算，最终可以最小化损失函数，进而获得预测器f

但是，如果矩阵$\mathbf{A}^\top\mathbf{A}$不可逆，我们应该如何最小化损失函数呢？

考虑矩阵不可逆的情况，这说明了样本数量小于特征维度，也就是说，此时预测器获得的信息不足以构建出整个模型，考虑人为增加一些条件。即，通过假设的方式人为“增加样本的数量”

“假设”，在贝叶斯统计中，可以体现为prior distribution，从而，我们可以运用贝叶斯统计的理论来解决这个问题

贝叶斯统计(from 卓致用)

我们要构建这个预测器$f:\mathbb{X}\to\mathbb{Y}$，可以通过 最大后验估计/Maximum A Posterior 的方式
由于后验概率$p\left(\beta \mid D\right) \propto p(D|\beta)p(\beta)$，所以

$$\max_{\beta} p(D|\beta)p(\beta) = \max_{\beta} \log p(D|\beta) + \log p(\beta)$$

$$\hat{\beta}_{\text{MAP}} = arg\max_{\beta} \underbrace{\log p\left(\{(X_i, Y_i)\}_{i=1}^{n} \mid \beta, \sigma^2\right)}_{\text{log likelihood}} + \underbrace{logp\left(\beta\right)}_{\text{log prior}}$$

第一行的含义是，我们要最大化数据的似然函数和参数的先验概率的乘积。

- 似然函数表示数据（D）在给定参数 β 的情况下出现的概率
- 先验概率表示我们在看到数据之前对参数的信念。

最大化这个乘积等价于最大化它们的对数，因为对数是单调递增的函数。

第二行的第一个项代表的含义是，在给定参数 β 和方差 σ^2 的情况下，数据出现的概率。

> 在统计模型中：
>
> - β 通常代表模型的系数
> - σ^2 代表模型中的噪声或误差的方差。即使我们有了 β，我们还需要知道数据中的变异性（或不确定性）有多大，这就是为什么 σ^2 是重要的。
>
> 在某些模型，比如线性回归中，我们假设数据 Y 是由自变量 X 的线性组合（由 β 确定）加上一些随机噪声（由 σ^2 描述）生成的。这个噪声代表了除了 X 影响 Y 之外的其它因素。所以，σ^2 帮助我们了解除了主要效应（由 β 描述）之外，数据中还有多少随机波动。

我们采取高斯先验，也即假设参数遵循高斯分布（也称为正态分布）。那么也就有
$\beta \sim N(0, \tau^2 I)$。

这表示：

- 参数 β 是一个 N 维向量，其中 N 是特征的数量
- 其遵循均值为 0，方差为 τ^2 的多元高斯分布
- $\tau^2 I$ 是指协方差矩阵为 τ^2 的对角矩阵，这代表 β 的每个元素都是相互独立的，且每个元素的方差都是 τ^2。
- 也就是说，各个维度独立同分布（iid）。

β 的概率密度分布函数按照矩阵表示，则是：

$$p(\beta) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(\beta-\mu)^T\Sigma^{-1}(\beta-\mu)}$$

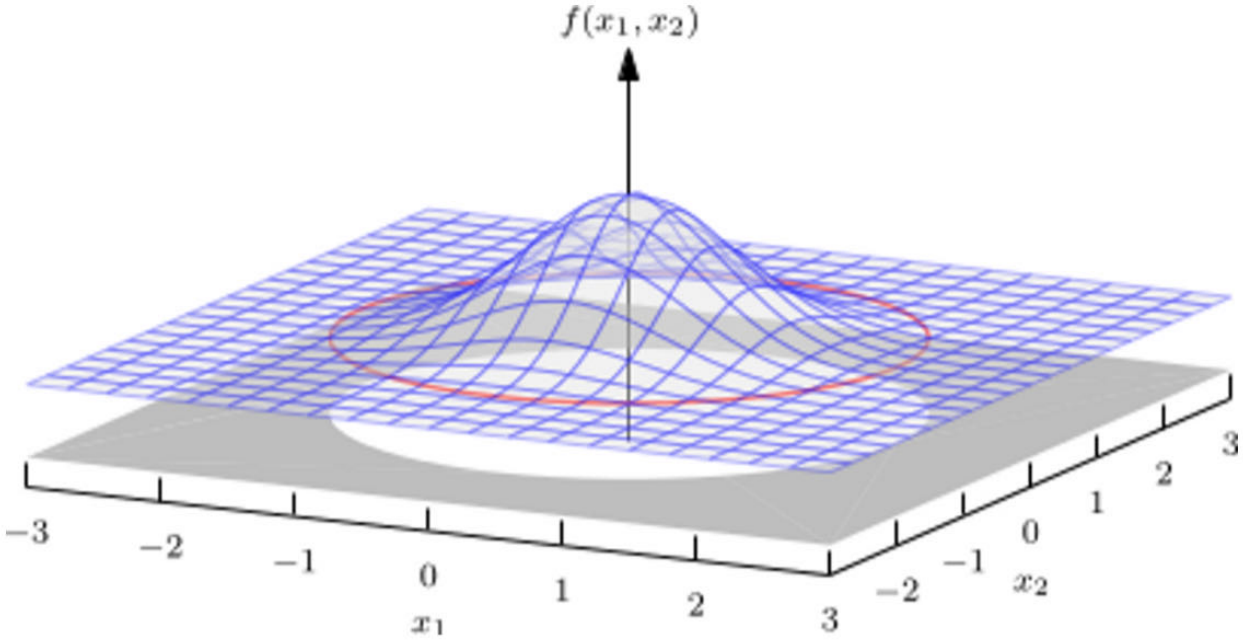
> 可以比较一维版本来理解~
>
> 其中的 $|\Sigma|$ 是协方差矩阵的行列式，它的作用是保证概率密度函数的总面积为 1。而当它为对角矩阵时，还能额外保证各个维度彼此独立。

所以，我们有 $p(\beta) \propto e^{-\beta^T\beta/2\tau^2}$。

这不包括归一化常数。$\propto$ 表示成比例，意味着这是未归一化的概率密度，它的形状随着 β 的变化而变化，但是总面积（概率总和）是固定的。$\beta^T\beta$ 是 β 的二次项，表示参数向量的长度的平方。

关于这一部分，可以阅读 [钱默吟/多元高斯分布完全解析]，讲的很详细。

这个式子的一个直观理解，就是假设 β 是一个二维的，那么我们可以可视化这个分布：



那么在这个图中，任意一个点都代表一个高斯分布，而 $f(x_1, x_2)$ 就是这个点的概率密度。我们可以看到，这个分布是关于原点对称的，这也是 $p(\beta)$ 中的 $\beta^T\beta$ 的作用，这代表了我们对于变量的顺序是没有偏好的。

由于$p(\beta) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} e^{-\frac{1}{2}(\beta-\mu)^{T\Sigma^{-1}(\beta-\mu)}} \propto e^{-\frac{1}{2}(\beta-\mu)^T\Sigma^{-1}(\beta-\mu)}$ 所以，$\log p(\beta) \propto \| \beta \|_2^2$

$$\hat{\beta}_{\mathbf{MAP}} = arg\min \sum_{i=1}^{n} (Y_i - X_i\beta)^2 + \lambda \| \beta \|_2^2$$

为什么刚才是 $arg\max$，现在又变成了 $arg\min$？简单说明就是，刚才我们要最大化从已知的 β 中得到数据真实分布的概率，这就等价于最小化使用我们的模型，从 X 中得到的数据的误差（也即最大似然估计）。

而 λ 是一个超参数，它的作用是控制我们对于参数的偏好，也就是我们对于模型的复杂度的偏好。在后续的学习中，我们会知道 $\lambda \| \beta \|_2^2$ 其实是一个正则项，它的作用是防止过拟合。

当我们对于 β 的先验假设不一样时，这个惩罚项（正则项）也会不一样。但大致原理不变，于是我们对这个形式加以推广，得到了类似如下的公式：

$$min_\beta (\mathbf{A}\beta - \mathbf{Y})^T(\mathbf{A}\beta - \mathbf{Y}) + \lambda \text{pen}(\beta) = minJ(\beta) + \lambda \text{pen}(\beta)$$

其中，$J(\beta)$ 是我们的损失函数，$\text{pen}(\beta)$ 是惩罚项。

对于惩罚项，我们一般有两个选择：

- L1 正则（1-范数）：$\text{pen}(\beta) = \| \beta \|_1 = \sum_{i=1}^{n} |\beta_i|$
- L2 正则（2-范数）：$\text{pen}(\beta) = \| \beta \|_2^2 = \sum_{i=1}^{n} \beta_i^2$

最终，我们得到**广义的线性模型**，我们可以考虑任意一种单调可微的函数 $g(\cdot)$：

$$y = g^{-1}(w^T x + b)$$

其中，$g(\cdot)$ 是激活函数，它的作用是将线性模型的输出转换为我们想要的输出，也即将线性模型的输出转换为一个非线性的预测值。比如，当 $g(\cdot)$ 是恒等函数时，我们就得到了线性回归；当 $g(\cdot)$ 是 sigmoid 函数时，我们就得到了逻辑回归。

02-机器学习-线性模型Part02

- Logistic Regression/Classification

Question

如何用线性模型将X分类到离散labels Y?

- 找一个单调可微的函数将Y和预测值联系起来
- **Unit-step function.**

$$y(z) = \begin{cases} 0, & z < 0 \\ 0.5, & z = 0 \\ 1, & z > 0 \end{cases}$$

- **Sigmoid (Logistic) function.**

$$y(z) = \frac{1}{1 + e^{-z}}$$

其中$z = w^T x + b$，从而：

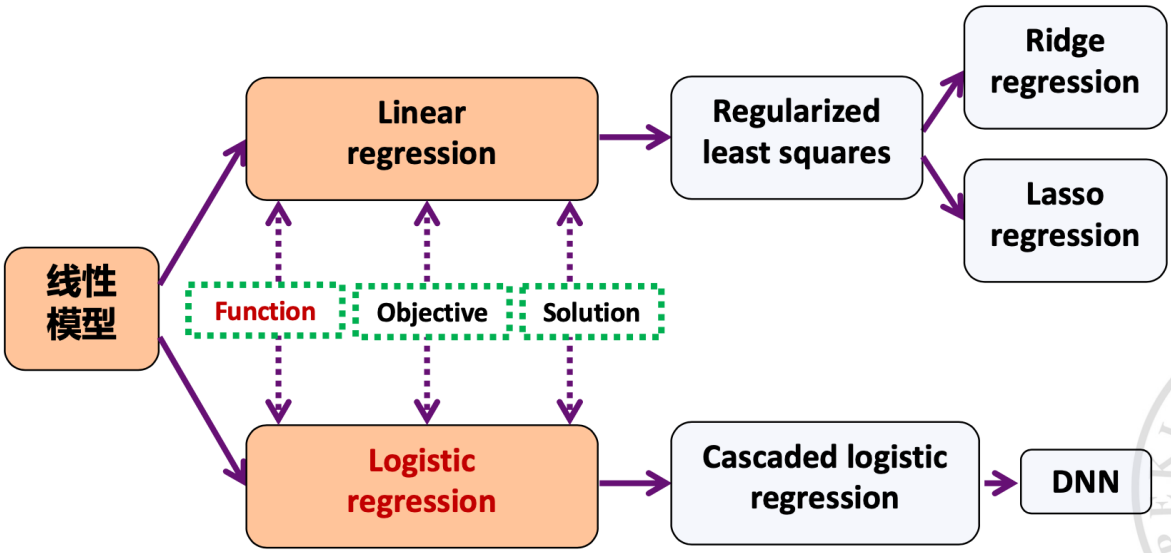
$$y = \frac{1}{1 + e^{-(w^T x + b)}}$$

也可以表示为：

$$\log \frac{y}{1 - y} = w^T x + b$$

Note

$$\frac{\partial sigmoid(x)}{\partial x} = sigmoid(x)\,(1 - sigmoid(x))$$



训练步骤：

Step1. 给定$\mathbf{x}$估计其出现$C_1 \mid x$的概率$P_{w,b}(C_1 \mid \mathbf{x}) = f_{w,b}(\mathbf{x})$，其中$f_{w,b}(\mathbf{x}) = \sigma\left(\sum_i w_i x_i + b\right)$

Step2. 找出目标函数/Objective function
给定训练数据

$$\begin{array}{ccccc} x^1 & x^2 & x^3 & \cdots\cdots & x^N \\ C_1 & C_1 & C_2 & \cdots\cdots & C_1 \end{array}$$

假设这些数据是基于$P_{w,b}(C_1 \mid \mathbf{x}) = f_{w,b}(\mathbf{x})$生成的
那么给定$\mathbf{w}$与$\mathbf{b}$，生成这些数据的概率是：

$$\mathcal{L}(w,b) = f_{w,b_r}(x^1) f_{w,b}(x^2)\left(1 - f_{w,b}(x^3)\right)\cdots f_{w,b}(x^N)$$

所以最优的$\mathbf{w}^*, \mathbf{b}^*$是：

$$w^*, b^* = \arg\max_{w,b} \mathcal{L}(w,b)$$

记label $\hat{y}$ 满足

$$\hat{y} = \begin{cases} 1, & C_1 \\ 0, & C_2 \end{cases}$$

所以

$$w^*, b^* = arg\max_{w,b} \mathcal{L}(w,b) = arg\min_{w,b} -log\mathcal{L}(w,b)$$

从而

$$\begin{aligned} -log\mathcal{L}(w,b) &= -\left(log f_{w,b}(x^1) + log f_{w,b}(x^2) + log\left(1 - f_{w,b}(x^3)\right)\cdots + log f_{w,b}(x^N)\right. \\ &= \underbrace{\sum_n -\left[\hat{y}^n log f_{w,b}(x^n) + (1-\hat{y}^n) log\left(1 - f_{w,b}(x^n)\right)\right]}_{\text{Cross Entropy}} \end{aligned}$$

Important

上面目标函数的负对数恰好是交叉熵/Cross Entropy

- 假设现在一个样本集合中有两个概率分布p与q，其中p代表真实分布，q代表非真实分布
- 用p衡量一个样本所需要的编码长度的期望为Cross Entropy：

$$H(p) = \sum_i p(i) \log(\frac{1}{q(i)})$$

- 对于离散变量：

$$H(p) = \sum_i p(i) \log(\frac{1}{q(i)})$$

- 对于连续变量：

$$H(p) = -\int_X P(x) \log Q(x) \mathrm{d}x$$

Step3. 梯度下降
计算梯度

$$\begin{aligned}
\frac{\partial(-log\mathcal{L}(w,b))}{\partial w_i} &= \sum_n -\left[\hat{y}^n \frac{\partial}{\partial w_i} log f_{w,b}(x^n) + (1-\hat{y}^n)\frac{\partial}{\partial w_i} log\left(1 - f_{w,b}(x^n)\right)\right] \\
&= \sum_n -\left[\hat{y}^n(1-f_{w,b}(x^n))x_i^n + (1-\hat{y}^n)f_{w,b}(x^n)x_i^n\right] \\
&= \sum_n -\left(\hat{y}^n - f_{w,b}(x^n)\right)x_i^n \\
&\propto \text{difference between label and prediction}
\end{aligned}$$

更新weight

$$w_i \leftarrow w_i - \eta \sum_n -\left(\hat{y}^n - f_{w,b}(x^n)\right)x_i^n$$

Question

为什么不用Square Error Loss Function?

如果用Square Error：

$$\mathcal{L}(f) = \frac{1}{2}\sum_n \left(f_{w,b}(x^n) - \hat{y}^n\right)^2$$

求梯度：

$$\frac{\partial \mathcal{L}}{\partial w_i} = (f_{w,b}(x) - \hat{y}) f_{w,b}(x) \left(1 - f_{w,b}(x)\right) x_i$$

无论$f_{w,b}(x^i) = 1/0$ 梯度都是0

> Logistic Regression局限性：
>
> - 无法处理多分类问题
> - 无法处理非线性问题

基于线性分类器的图片分类

图片分类：Computer Vision中的一个重要问题

- 问题：给定一张图片，判断它属于哪个类别
- 输入：图片
  - 一张图片可以看作是一个像素tensor
    - 例如 $32 \times 32 \times 3$ (RGB 3 channels)
- 输出：类别
- 任务：分类问题
- 挑战：
  - 图片的角度不一
  - 图片的光照不一
  - 图片的背景不一
  - 图片的遮挡不一
  - 图片的噪声不一

我们想要设计一个hard-code的算法来解决这个问题，但是这个问题太复杂了，我们无法hard-code所有的规则，所以我们需要使用data-driven的方法来解决这个问题，也就是机器学习的方法。

- 机器学习：数据驱动的方法
  - 收集图片和标签的数据集
  - 使用机器学习算法训练一个分类器
  - 在测试图片上评估此分类器

线性分类器/Linear Classifier

线性分类器是一个典型有参数的分类器，它的输出是输入的线性组合，然后通过一个阈值函数来判断输出的类别。

例如，对于一个$32 \times 32 \times 3$的图片，我们可以将其展开为一个$32 \times 32 \times 3 = 3072$维的行向量，然后我们可以找到一个最优的参数$W$和$b$，使得$xW + b$能够最好的区分不同的类别。

如果我们给定10个标签，将$32 \times 32 \times 3$的图片分类到这10个labels中，那么我们可以得到一个3072×10的矩阵W和一个10×1的向量b。

预测器：$f: \mathbb{R}^{3072 \times 1} \to \mathbb{R}^{10 \times 1}$满足$f(x, W) = xW + b$

那么，我们该如何找到这个Linear Classifier的最优参数W^*与b^*呢？

- 定义一个可以衡量训练数据上预测标签和真实标签差异的loss function
- 给定train dataset $\{(x_i, y_i)\}_i^N$
- 求解

$$W^*, b^* = \arg\min_{W,b} \sum_{i=1}^{N} \mathcal{L}(f(x_i, W), y_i)$$

多项式逻辑回归分类器/Multinomial Logistic Regression

我们从原始的线性分类器$f(x, W) = xW + b$得到一个"score vector"，但是这个score vector并不是一个概率分布，我们需要将其转换为一个概率分布。

使用Softmax函数：

$$P(Y = k \mid X = x_i) = \frac{\exp(s_k)}{\sum_j \exp(s_j)}$$

注意：向量中的分量有可能很大，导致指数爆炸问题，所以我们需要预处理向量
$x_i \leftarrow x_i - \max_j \left\{x_i^j\right\}$

由于

$$\frac{\exp(x_i^k - \max_j \left\{x_i^j\right\})}{\sum_k \exp(x_i^k - \max_j \left\{x_i^j\right\})} = \frac{\exp(s_k)}{\sum_j \exp(s_j)}$$

所以这样做是合理的

求得概率分布y_{pred}与真实标签y_{true}后，我们需要找到合适的Loss Function来衡量差异：

在这里，我们选择Cross Entropy Loss Function：

$$\mathcal{L}(y_{\text{pred}}, y_{\text{true}}) = -\sum_k y_{\text{true}}^k \log y_{\text{pred}}^k$$

最近邻分类器/Nearest Neighbor Classifier

K-近邻分类器是一种无参的方法，但是仍然是有监督学习，有训练数据

训练数据：$\{(x_i, y_i)\}_{i=1}^N$

预测器：$f: \mathbb{R}^{3072 \times 1} \to \mathbb{R}^{10 \times 1}$

训练：存储训练数据

预测：对于一个新的输入x，找到训练数据中与x最相似的k个数据点，然后返回这k个数据点中出现最多的标签

那么，什么叫做“最相似”呢，我们需要定义一个距离函数来衡量两个数据点之间的相似度，通常我们使用以下几种Distance Metric：

- L_1距离：

$$d(x_i, x_j) = \sum_k |x_i^k - x_j^k|$$

- L_2距离：

$$d(x_i, x_j) = \sqrt{\sum_k (x_i^k - x_j^k)^2}$$

其中，k与Distance Metric依赖于我们的选择，这种需要自行选择的参数称为 超参数/Hyperparameter

- 参数：在训练过程中需要学习的参数
  - 例如：W与b
- 超参数：在训练之前需要选择的参数
  - 例如：k与Distance Metric
  - 依赖于数据集与任务

聚类/Clustering

聚类是一种无监督学习，没有标签的数据集，目标是找到数据集中的一些内在的结构，把相似的数据聚到一起

聚类的特点是：

- 高类内相似度
- 低类间相似度

我们如果想要将数据集中的数据聚到一起，我们需要定义一个相似度/距离的度量，然后我们可以使用K-Means算法来实现这个目标

Distance Metric:

- L_1距离

$$d(x, y) = \sum_k |x^k - y^k|$$

- L_2距离

$$d(x,y)=\sqrt{\sum_k (x^k-y^k)^2}$$

- L_∞距离

$$d(x,y)=\max_k |x^k-y^k|$$

聚类算法：

- 分割方法/Partition algorithm
  - K-Means
  - Mixture-Model based clustering
- 层级方法/Hierarchical algorithm
  - Single-linkage
  - Average-linkage
  - Complete-linkage
  - Centroid-based

分割算法：把n个对象分割成K个簇（Clusters），使得每个对象属于且仅属于一个组。

目标：最优化某个选定分割标准的K组分割。

K-means是一种常用的聚类算法，旨在将数据集分成K个簇，使得同一个簇内的数据点之间距离尽可能小，而不同簇之间的数据点距离尽可能大。算法步骤如下：

算法步骤：

1. 随机选择K个数据点作为初始簇中心。(随机选择的K个数据点即为“种子”)
2. 将每个数据点分配给最近的簇中心，形成K个簇。
3. 重新计算每个簇的中心，即每个簇内所有数据点的均值。
4. 重复步骤 2 和 3，直到簇中心不再变化或变化非常小。

其他问题：

1. 对于种子的选择敏感，需要尝试不同的初始种子，且尽量让k个初始种子互相远离。
2. 受离群值（Outliers）影响较大。

神经网络基础

单个神经元

对于单个神经元来说，它有1个输出z与n个输入$\{x_i\}_{i\geq 1}^n$，输出是输入的 线性组合/linear combination

$$z = \sum_{i=1}^{n} w_i x_i$$

- 一个神经元是只有一个输出的单层神经网络
- 因为输出与所有输入连接，这层神经网络可以被称为 全连接层/full connected layer or dense layer

对于神经元的计算，我们可以使用 向量/vector 来表示，通常有列格式与行格式两种表示方法

列格式（常见于书籍）：
$w = (w_1 \quad w_2 \quad \cdots \quad w_n)^\top, x = (x_1 \quad x_2 \quad \cdots \quad x_n)^\top, z = w^\top x$
行格式（常见于代码）：
$w = (w_1 \quad w_2 \quad \cdots \quad w_n)^\top, x = (x_1 \quad x_2 \quad \cdots \quad x_n), z = xw$

偏置/bias：$b \in \mathbb{R}$使得输出值可以上下浮动，以更好的适应输入值

$$z = \sum_{i=1}^{n} w_i x_i + b$$

```
import torch
# Single neuro

# matrix multiplication 矩阵乘法
x = torch.tensor([[1., 2., 3.]])          # 1x3
w = torch.tensor([[-0.5], [0.2], [0.1]]) # 3x1
print("X:\n", x, "\nShape:", x.shape)
print("W:\n", w, "\nShape", w.shape)

# Bias 偏置
b1 = torch.tensor(0.5)

# 计算有偏置的矩阵乘法
z1 = torch.matmul(x, w) + b1
print("Z:\n", z1, "\nShape", z1.shape)
```

对于分类问题，偏置可以改变决策平面的位置，从而使得分类器更加灵活

- 两个输入特征:决策边界是一条线
- 三个输入特征:决策边界是一条平面
- 更多个输入特征:决策边界是一条 超平面/hyperplane

若两个神经元连接在一个（3 inputs - 2 outputs）

$$(z_1 \quad z_2) = (x_1 \quad x_2 \quad x_3) \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{pmatrix} + (b_1 \quad b_2)$$

```
# Two outputs 两个输出
x = torch.tensor([[1., 2., 3.]]) # 1x3
w = torch.tensor([[-0.5 ,  -0.3],
                                  [0.2  ,  0.4],
                              [0.1  ,  0.15]]) # 3x2
print("X:\n", x, "\nShape:", x.shape)
print("W:\n", w, "\nShape", w.shape)

# Bias 偏置
b2 = torch.tensor([0.5, 0.4])
z2 = torch.matmul(x, w) + b2
print("Z:\n", z2, "\nShape", z2.shape)
```

激活函数

激活函数是神经网络中的一个重要组成部分，它的作用是引入非线性性，使得神经网络可以拟合更加复杂的函数

sigmoid函数

给定一个神经网络，输出可以表示分数，比如踢足球的概率。为了表达0的概率，需要把输出值限制到0 1之间。

$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

```
z1 = torch.Tensor([[0.7000]]) # 线性组合值
# Activation function 激活函数
# Sigmoid function
sigmoid_torch = torch.nn.Sigmoid()
a1 = sigmoid_torch(z1)
print("Result torch sigmoid:", a1)

# define your own activation function
class ActSigmoid(torch.nn.Module):
    def __init__(self):
        super(ActSigmoid, self).__init__()
    def forward(self, x):
        return 1/(1+torch.exp(-x))

sigmoid_act = ActSigmoid()
a1_m = sigmoid_act(z1)
print("Result your own sigmoid:", a1_m)
```

tanh函数

与Sigmoid函数类似，Hyperbolic tangent(tanh)函数也是限制输出值到一个范围。但Tanh函数限制的范围是 -1~1，该函数往往用于回归任务(regression task)，比如网络输出图像，而图像的像素数值在 -1~1之间;再比如输出情绪，-1代表不开心，1代表开心，0代表中间状态。

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 1 - \frac{2}{1 + e^{2z}}$$

ReLU函数

ReLU函数把负值输出设为0，正值输出不变。可用作特征选取(feature selection)和简化网络优化(optimisation)计算(后文会讲解)

$$f(z) = \max(0, z) = \begin{cases} 0, \text{if } z \leq 0 \\ z, \text{if } z > 0 \end{cases}$$

Leaky ReLU函数

然而把负输出设为0会丢失输入信息导致，一个解决方案是采用Leaky ReLU函数。通过一个很小 是正数α(如0.1、0.2)来控制斜率(slop)，使得负输出的信息可以在激活值上体现。此外，Parametric ReLU(有参数的ReLU，也称PReLU)将α当作一个网络的参数，在后续优化过程中自行学习调整斜率。

$$f(z) = \begin{cases} \alpha z, \text{if } z \leq 0 \\ z, \text{if } z > 0 \end{cases}$$

Softmax函数

一个网络有多个输出的时候，我们可以输出多个情况下的概率，实现多分类任务 (multi-class classification)，例如输出选择踢足球、打篮球、跑步的概率。Softmax 是用在多分类任务下网络输出层上的函数，用以让所有激活值表示0~100%的概率， 并让所有激活值之和为1，则100%。

给定一个输入向量$z = (z_1 \quad z_2 \quad \cdots \quad z_n)$，向量维度是$n$，输出向量

$$softmax(z)^k = \frac{e^{z_k}}{\sum_{i=1}^{n} e^{z_i}}$$

```
# Softmax
z2 = torch.Tensor([[0.7000, 1.3500]])
softmax_torch = torch.nn.Softmax(dim=1)
a2 = softmax_torch(z2) # z2是神经网络未激活的值
print("Result torch softmax:", a2)

# define your own activation function
class ActSoftmax(torch.nn.Module):
        def __init__(self):
                super(ActSoftmax, self).__init__()
        def forward(self, x):
                return torch.exp(x)/torch.sum(torch.exp(x), dim=1, keepdim=True)
```

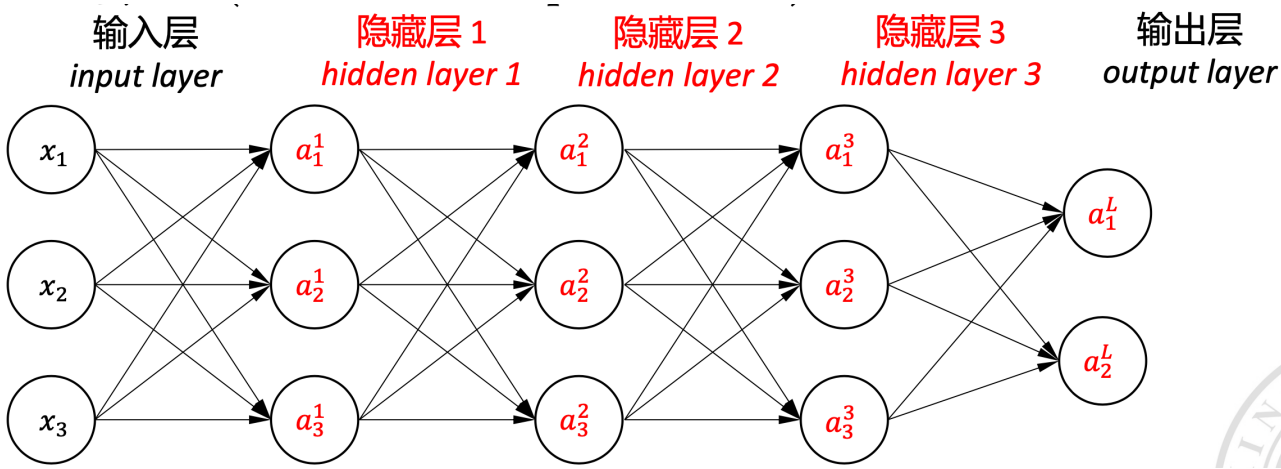
```
softmax_act = ActSoftmax()
a2_m = softmax_act(z2)
print("Result your own softmax:", a2_m)
```

多层感知器

多层感知器(multi-layer perceptron，MLP)在单层全连接网络上拓展，至少有2层全连接层。

在原有层上堆叠新的层，可以被认为是将原有层的输出值当作特征值来学习

因此，和单层全连接网络相比，MLP可以处理更复杂的输入数据，具有更好的 表达能力/representation capacity



损失函数

目的：

1. 损失函数/loss function 用来量化网络 预测的输出/predicted output 和 给定训练数据输出/ground truth 之间的 误差/error or loss value
2. 损失函数用来设定优化神经网络参数(如权重、偏置)的目标
3. 优化神经网络的过程，是通过更新其参数来使得误差尽可能小
4. 梯度下降/gradient descent 是最常用的优化方法，后文将有介绍

逻辑回归损失/Logistic Regression Loss

二值分类：

$$f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$

M个数据样本：

$$\mathcal{L} = \sum_{m=1}^{M}(y^m\, log(a^m) + (1-y^m)\log(1-a^m))$$

交叉熵损失函数/Cross-Entropy Loss

M个数据样本：

$$\mathcal{L} = \sum_{m=1}^{M} y_k^m \log(a_k^m)$$

```
output = mlp(x_input)# 前向传播

# 多分类交叉熵
target = torch.tensor([[0, 1, 0]],dtype=torch.float) # 1x3
print("Target value:{}".format(target.numpy()))
print("Neural network output:{}".format(output.detach().numpy()))
loss_fn = torch.nn.CrossEntropyLoss()
l_ce = loss_fn(output, target)
print("Loss cross entropy:", l_ce)
```

L_p范数/norm

$$\|x\|_p^p = \sum_{k=1}^{K} |x_k|^p$$

均方误差/Mean Squared Error

$$\mathcal{L}_{MSE} = \frac{1}{M} \sum_{m=1}^{M} \|\mathbf{y}^m - \mathbf{a}^m\|_2^2$$

平均绝对误差/Mean Absolute Error

$$\mathcal{L}_{MAE} = \frac{1}{M} \sum_{m=1}^{M} |y^m - a^m|$$

```
# Mean absolute error 平均绝对误差
def mae(output, target):
        return torch.mean(torch.abs(output - target))
# Mean squared error 平均平方误差
def mse(output, target):
        return torch.mean((output - target)**2) y1 = torch.tensor([[1., 3., 5.,
7.]])
y2 = torch.tensor([[2., 4., 6., 8.]])
# 计算MAE和MSE
l_mae = mae(y1, y2)
l_mse = mse(y1, y2)
print("Loss MAE: {} \nLoss MSE: {}".format(l_mae, l_mse))
mse_torch = torch.nn.MSELoss()
l_mse_torch = mse_torch(y1, y2)
print("Loss MSE torch: {}".format(l_mse_torch))
```

优化

目的:

给定网络结构$f(x;\theta)$ 、定义好的损失函数$\mathcal{L}$和训练数据样本，训练网络的过程是通过更新网络参数θ(比如权重、偏置)以最小化误差的过程。

梯度下降(Gradient descent)是一个更新网络参数的方法，此外依然有很多优化方法存在，比如BFGS (L-BFGS) 和 conjugate gradient (CG)，但这些方法往往需要很多计算量，因此很少使用。

梯度下降法/Gradient Descent

$$\theta := \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$$

其中α是学习率/learning rate，是一个超参数，需要手动调整

误差反向传播/Back propagation

误差反向传播是用来计算网络中所有参数的梯度$\frac{\partial \mathcal{L}}{\partial \theta}$的方法。计算梯度时，通过引入中间结果$\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$，利用 链式法则/Chain Rule 可以逐层向前（反向！）计算梯度

算法（行格式）

Notation. $\circ$为Hadamard product，逐点相乘

1. 已知:

$$\begin{aligned}
\mathbf{a}^l &= f(\mathbf{z}^l) = \frac{1}{1+e^{-\mathbf{z}^l}} \\
\mathbf{z}^l &= \mathbf{a}^{l-1}\mathbf{W}^l + \mathbf{b}^l \\
\mathcal{L} &= \frac{1}{2}(\mathbf{y} - \mathbf{a}^l)^2
\end{aligned}$$

2. 求导:

\begin{align*} \frac{\partial \mathbf{a}^{l}}{\partial \mathbf{z}^{l}} &= f'(\math

\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{l}}&= \mathbf{a}^{l} - \mathbf{y}\\

\frac{\partial z^{l}}{\partial \mathbf{W}^{l}}&= \mathbf{a}^{l-1}~\mathrm{and}~\frac{\partial z^{l}}{\partial b^{l}}=1
\end{align*}

3.输出层的中间结果:

\delta^{l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{l}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{l}}\frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^{l}}=(\mathbf{a}^{l} -

\mathbf{y})\circ(\mathbf{a}^{l}\circ(1-\mathbf{a}^l))$$
4. 其他层的中间结果

$$\delta^l = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^l} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{l+1}} \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l} = \delta^{l+1} \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l}$$

$$\mathbf{z}^{l+1} = \mathbf{a}^l \mathbf{W}^{l+1} + \mathbf{b}^{l+1}$$

$$\frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{z}^l} = \frac{\partial \mathbf{z}^{l+1}}{\partial \mathbf{a}^l} \frac{\partial \mathbf{a}^l}{\partial \mathbf{z}^l} = \mathbf{W}^{l+1} \circ (\mathbf{a}^l \circ (1 - \mathbf{a}^l))$$

$$\delta^l = \delta^{l+1} \mathbf{W}^{l+1} \circ (\mathbf{a}^l \circ (1 - \mathbf{a}^l))$$

5. 计算梯度

\begin{align*} \frac{\partial\mathcal{L}}{\partial \mathbf{W}^{l}}&= \frac{\partia

\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{l}} &= \frac{\partial\mathcal{L}}{\partial \mathbf{z}^{l}}\frac{\partial \mathbf{z}^{l}}{\partial \mathbf{b}^{l}} =\delta^l
\end{align*}$$

6. 更新参数

\begin{align*} \mathbf{W}^{l}&:= \mathbf{W}^{l} - \alpha\frac{\partial\mathcal{L}}{

\mathbf{b}^{l}&:= \mathbf{b}^{l} - \alpha\frac{\partial\mathcal{L}}{\partial \mathbf{b}^{l}}
\end{align*}$$

正则化

实现